

The CARP Mobile Sensing Framework – Demonstration

Jakob E. Bardram

jakba@dtu.dk

Department of Health Technology,
Technical University of Denmark
Kgs. Lyngby, Denmark

Nigel Sjölin Grech

nsjgr@dtu.dk

Department of Health Technology,
Technical University of Denmark
Kgs. Lyngby, Denmark

Abstract

Mobile sensing – i.e., the ability to unobtrusively collect sensor data from built-in phones and attached wearable sensors – is a core research topic in Ubicomp. Here, we present a demonstration of the CARP Mobile Sensing (CAMS) framework, a modern cross-platform (Android / iOS) software architecture providing a reactive and unified programming model that emphasizes extensibility, maintainability, and adaptability. CAMS has evolved since 2018, released in several versions, has been used in implementing a wide number of mobile health (mHealth) applications, and is well-documented. We demonstrate the software design, the programming model, documentation, and extensibility of CAMS.

CCS Concepts

• **Human-centered computing** → Ubiquitous and mobile computing systems and tools; • **Software and its engineering** → Development frameworks and environments; • **Hardware** → Sensor applications and deployments; • **Information systems** → Open source software; • **Applied computing** → Health informatics.

Keywords

mobile sensing; wearable sensing; context-aware computing; mobile health; mHealth; digital phenotyping; sensors; Flutter

ACM Reference Format:

Jakob E. Bardram and Nigel Sjölin Grech. 2025. The CARP Mobile Sensing Framework – Demonstration. In *Companion of the 2025 ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp Companion '25)*, October 12–16, 2025, Espoo, Finland. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3714394.3754442>

1 Introduction

Since the birth of the smartphone, research in mobile sensing has been a core topic in the Ubicomp community [12]. A significant body of research has been applying mobile sensing to e.g., health and wellness applications [5], including, for example, the EmotionSense [14], BeWell [13], and StudentLife [25].

Several general-purpose frameworks and platforms for mobile sensing include support for the configuration of sampling protocols, accessing low-level sensor data, and handling and storing this

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

UbiComp Companion '25, Espoo, Finland.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1477-1/25/10

<https://doi.org/10.1145/3714394.3754442>

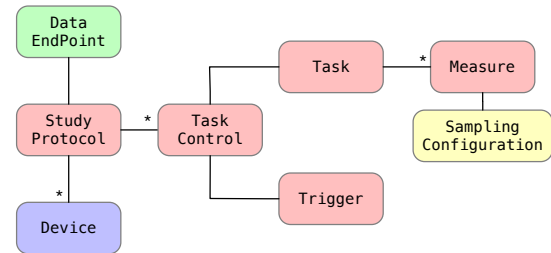


Figure 1: The CAMS StudyProtocol domain model.

data have been proposed [10]. Most of this research has focused on providing easy-to-use *platforms* for the collection of data from mobile phones and storing this in a cloud-based infrastructure. These platforms typically have the options to configure a sampling ‘study’, enroll a set of participants, deploy the study onto the participants’ mobile phones, and automatically collect data in a cloud infrastructure, which can be accessed from a web portal. Contemporary examples of this approach include Purple Robot [22], Sensus [26], the AWARE Framework [8], the Beiwe Research Platform [23], mCerebrum [9], RADAR-base [20], and LAMP [24]. These platforms are designed for research and target experimental behavioral researchers as end-users; the goal is to allow researchers to easily configure a study, enroll participants, deploy the study on the participants’ phones, and collect the data automatically.

However, we have found that often there is a need for designing application-specific apps for particular domains and patient groups, and to be able to add support for mobile and wearable sensing to such apps. Often, the motivation for participants to engage in these studies relies on that ‘there’s something in it for them’ [6], which again means that the app should be designed with the participant in mind – not the researcher.

Therefore, there is a need for a mobile and wearable sensing *programming framework* that allows adding data sampling to a domain-specific app during design and implementation. For this purpose, we have designed, implemented, evaluated, and released the CARP Mobile Sensing (CAMS) programming framework [2]. This framework is part of the overall Copenhagen Research Platform (CARP) and has been evolving over thirteen major releases, and has been used in the design and implementation of several released mHealth apps for mental health, cardiovascular diseases, and diabetes (see Table 1 for an overview). CAMS is a state-of-the-art mobile sensing platform with support for the collection of a wide range of data types. However, compared to other sensing platforms, CAMS has a set of unique design goals and novel contributions:

Cross-platform – CAMS supports a unified programming application programming interface (API) and programming

language for both Android and iOS programming. Hence, the same code base and programming API is used to develop apps for both Android and iOS.

Unified sensing – CAMS has a unified sensing concept for both on-board and off-board (e.g., wearable) sensing. This unified concept also cuts across ‘passive sensing’, i.e., automatic collection of sensor data in the background, and ‘active sensing’, i.e., the collection of data from the user.

Data Management – CAMS has extensive support for different data sampling, transformation, storage, and upload methods. A uniform approach to data transformation is adopted, which supports both privacy-enabling transformations as well as transforming data to make it comply with official data standards, including medical data standards.

Extensible – CAMS is highly extensible in a number of ways: it allows for implementing new data sampling methods (including both phone sensing, external wearable devices, and online services); it supports creation of new data transformers (both for privacy reasons and data standards), and it allows for creating custom data managers, which can upload data in a specific format to specific servers (or other kinds of data offloading).

Maintainable – CAMS integrates with a publicly available online build and dependency management system, which allows for easy access to CAMS libraries and for sharing custom libraries amongst a CAMS programming community.

Reactive API – The CAMS programming API is designed to be simple, yet expressive for the programmer. It supports a modern reactive, stream-based programming model, allowing for non-blocking sensing and data processing. This is important to avoid interference of data sampling with the responsiveness of the app.

The basic scenario CAMS supports is to enable programmers to design and implement a custom cross-platform mHealth app, which primarily focuses on features for the patient. CAMS then enables programmers to add mobile sensing capabilities in a flexible and simple manner to the application-specific code base. This includes configuring the collection of health and behavioral data like electrocardiography (ECG), location, activity, and step count; formatting this data according to different health data formats; using this data in the app (e.g., showing it to the user); and uploading it to a server, using a specific API (e.g. REST), in a custom format. This scenario is what we intend to demonstrate at UbiComp 2025, and to help researchers get started using CAMS.

2 CAMS Application Programming Model

The main purpose of CAMS is to provide a unified and reactive programming API for cross-platform sensing. CAMS is implemented in Flutter, which is a cross-platform software development toolkit for building natively compiled mobile applications for iOS and Android. Flutter uses the Dart programming language, which is a modern object-oriented, reactive, non-blocking language.

2.1 Code Example

Listing 1 shows how sampling can be added to a Flutter app. This basic example illustrates how sampling is configured, initialized,

started, and used in four basic steps: (i) a study protocol is defined (line 1–17) that collects step, light, screen, and battery events from the phone and stores data locally in a SQLite database; (ii) the runtime environment is created and initialized with this study protocol (19–23); (iii) the stream of sampling measurements is consumed and used in the app (25–28); and (iv) sampling is started (31).

```

1 // Create a protocol collecting data from the phone.
2 final phone = Smartphone();
3 final protocol = SmartphoneStudyProtocol(
4   ownerId: 'AB',
5   name: 'Tracking steps, light, screen, and battery',
6   dataEndPoint: SQLiteDataEndPoint(),
7 )
8 ..addPrimaryDevice(phone)
9 ..addTaskControl(
10   ImmediateTrigger(),
11   BackgroundTask(measures: [
12     Measure(type: SensorSamplingPackage.STEP_COUNT),
13     Measure(type: SensorSamplingPackage.AMBIENT_LIGHT),
14     Measure(type: DeviceSamplingPackage.SCREEN_EVENT),
15     Measure(type: DeviceSamplingPackage.BATTERY_STATE),
16   ]),
17   phone);
18
19 // Create and configure a client manager for this phone.
20 await SmartPhoneClientManager().configure();
21
22 // Create a study based on the protocol.
23 await SmartPhoneClientManager().addStudyFromProtocol(
24   protocol);
25
26 // Listening to the measurements stream.
27 SmartPhoneClientManager().measurements.listen((
28   measurement) {
29     // Use the measurement in the app.
30   });
31
32 // Start sampling.
33 SmartPhoneClientManager().start();

```

Listing 1: Configuring phone sensing in CAMS.

2.2 Software Domain Model

The CAMS domain model and API are quite elaborate and are designed to encompass a wide range of use cases. CAMS and its programming model is thoroughly documented¹. Here we will merely provide a brief overview of important parts, especially the domain model of the study protocol, which defines what data to collect, and the overall architecture describing the overall runtime behavior of CAMS and how to extend it.

Fig. 1 shows the Study Protocol domain model, which specifies how a sampling protocol is defined. The StudyProtocol holds the entire definition of the study by defining which Devices to use and what DataEndPoint to store the data. Listing 1:1–8 shows how a protocol is configured to use one device (the phone) and the local SQLite database as an endpoint. Data collection is defined as a set of TaskControls, each having a Trigger and a Task, which lists a set of Measures to collect.

A Trigger defines when to start or stop a Task, and hence describes *when* to collect data. CAMS comes with a set of built-in triggers, including triggers that starts sampling immediately (the

¹See <https://github.com/cph-cachet/carp.sensing-flutter/wiki>

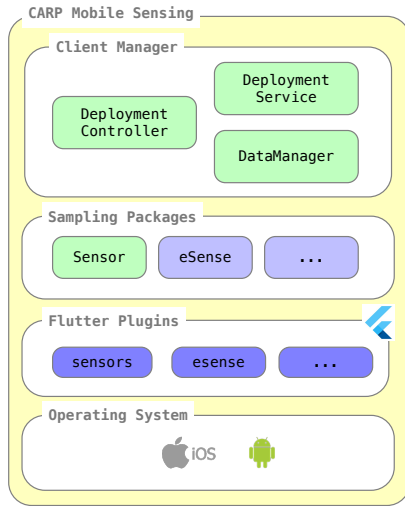


Figure 2: A simplified outline of the CAMS architecture.

ImmediateTrigger used in Listing 1:10), a trigger that runs periodically with a fixed period (PeriodicTrigger), a trigger that can be scheduled following a recurring pattern (the RecurrentScheduled Trigger), and a trigger that starts when a certain sampling event happens, such as when the phone enters a certain geofence (the Sampling EventTrigger).

A Task is a bundle of measures to be done simultaneously, once initiated by a trigger. For example, a task can specify to sample accelerometer and gyroscope data when triggered during a tremor assessment for Parkinson’s Disease. In Listing 1:11-16, a task is configured to sample step, light, screen, and battery events. CAMS comes with two built-in tasks. The BackgroundTask that runs automatically in the background and is used for passive sensing, and the AppTask that notifies the app when triggered and is used for active sensing, i.e., when asking for user input.

A Measure defines *what* to measure, i.e., the specific data stream. A measure is a configuration of the type of data to collect, which during runtime maps to a specific probe that can collect this type of data. Since CAMS follows a reactive programming model, all sampled data is collected in *streams* by listening to the underlying sensors. Measures often need detailed configuration, such as specifying the scanning frequency for Bluetooth, which can be defined by a SamplingConfiguration. However, this low-level configuration is often irrelevant to the user, and all measures thus come with default sampling configurations defined in a SamplingSchema.

2.3 CAMS Runtime Architecture

Fig. 2 shows a simplified overview of the layered software architecture of CAMS. The Client Manager controls all sensing, as also illustrated in Listing 1:19-31. The client manager comprises a set of services, controllers, and registries to handle deployment services, data managers, sampling packages, devices, and other related components. The Deployment Controller is responsible for obtaining a study protocol from a Deployment Service, executing the study protocol, transforming the collected data, and uploading it to the

specified Data Manager. Data is collected via a set of Sampling Packages, each specialized to collect a certain type of data from a device. For example, the ‘context’ package can collect location, weather, and activity data from the phone, while the ‘movesense’ package can collect data from a connected Movesense ECG device. Which sampling packages to use can be configured according to the specific application. For example, there is no need for sampling ECG in a diabetes app, and the Movesense package is hence not loaded. Each sampling package uses one or more Flutter plugins that access processes and services in the underlying operating system (OS), external wearable devices, or online services. For example, the health plugin accesses data from Apple Health on iOS and Google Health Connect on Android.

CAMS is designed to be highly configurable and extensible [3] in several ways. First, new sensing capabilities can be added by creating a sampling package. A sampling package specifies what Measures it can collect, what Device it uses for data collection, and a set of Probes that collect the data at runtime. Once a sampling package is implemented, its measures can be added to a protocol, and the data will be collected and uploaded together with the rest of the measures. Listing 2 shows how the eSense sampling package [1] enables adding an eSense device to the protocol and using this to collect two measures – button presses and sensor data from the eSense earbud (lines 8–14). Since the eSense earbud is connected via Bluetooth to the phone, we can also collect noise via the earbud by adding the NOISE measure to the protocol (lines 16–22).

```

1 // Define the eSense device and add its measures
2 ESenseDevice eSense = ESenseDevice(
3     deviceName: 'eSense-0332',
4     samplingRate: 10,
5 );
6 protocol.addConnectedDevice(eSense, phone);
7
8 protocol.addTaskControl(
9     ImmediateTrigger(),
10    BackgroundTask(measures: [
11        Measure(type: ESenseSamplingPackage.ESENSE_BUTTON),
12        Measure(type: ESenseSamplingPackage.ESENSE_SENSOR)
13    ]),
14    eSense);
15
16 // Collect noise every 20 secs for 5 secs
17 protocol.addTaskControl(
18     ImmediateTrigger(),
19    BackgroundTask(measures: [
20        Measure(type: MediaSamplingPackage.NOISE)
21    ]),
22    phone);

```

Listing 2: Using the eSense sampling package in CAMS.

Second, new data managers can be implemented and used for custom data storage or upload. For example, in the MUBS application [21], data was uploaded to Google Firebase using a Firebase data manager. A data manager listens to the stream of measurements collected and transforms them to a format that can be stored in the specific data backend while also taking care of runtime management, security, and authentication.

Third, new deployment services can be used. A deployment service is responsible for ‘deploying’ a study protocol, i.e., to create a configuration of a study protocol that fits the specific smartphone on which the protocol is to be executed. Right now, CAMS comes

Table 1: Examples of mHealth app build using CAMS.

Name	Domain	Sampling Packages
MUBS [21]	Mental Health	device, context
mCardia [11]	Cardiology	context, audio, movisens
DiaFocus [4]	Type-2 Diabetes	device, sensor, context, health, survey
m-Path Sense [15]	Experience Sampling	device, sensor, context, apps, connectivity, communication, audio
CARP Studies	Digital Phenotyping	all
mQoL Lab [7]	Quality of Life	sensor, context, survey
EFT [18]	Behavior Change	sensor, context, communication

with an on-device deployment service (the Smartphone Deployment Service) which is used in the example in Listing 1. However, study deployment can also be downloaded from a server (as supported in the CARP Deployment Service) or linked by a QR code.

2.4 Supported Measures

CAMS currently supports collecting a wide range of measures both from onboard and off-board (e.g., wearable) sensors². All of these measures are packaged in different sampling packages. Onboard measures include phone-based sensors, like accelerometer, gyroscope, GPS, and light; access to phone logs like phone, SMS, and calendar logs; collection of communication measures like wifi, Bluetooth devices, and beacons. Collection from local and online services includes access to Apple or Google Health, and weather and air quality services. Supported connected sensors include the Polar heart rate (HR) devices, the Movisens and Movesens ECG devices, and the eSense earbud devices. More device support is in the making, including support for the Dexcom and the Garmin devices.

2.5 Availability

CAMS and all of its components are available under a permissive MIT license at GitHub and are released in the Flutter ecosystem at pub.dev. This entails that all new CAMS components are available as part of the Flutter package manager infrastructure, and all CAMS programmers can hence easily share and use the different packages.

3 CAMS App Examples

CAMS have been used in designing, implementing, and deploying a set of mHealth applications (see Table 1). These applications have formed the basis for the continued re-design and extension of CAMS, which now supports a wide range of use cases. For example, the DiaFocus app [4] targets ambulatory treatment of type-2 diabetes. It collects a wide range of passive data (battery and screen events, location, activity, and weather), sensor data from connected glucose monitors via the Apple Health connection, and active data

²See <https://github.com/cph-cachet/carp.sensing-flutter/wiki/A.-Measure-Types> for a complete overview

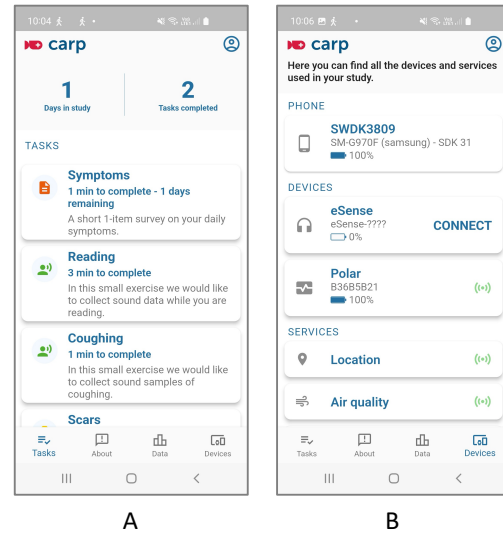


Figure 3: The CARP Studies app. A: List of App Tasks for the user to do (active sensing). B: Connected devices and services as defined by the Study Protocol.

from users on sleeping, smoking, diet, and alcohol, combined with a set of standardized diabetes-related questionnaires.

The CARP Studies app, on the other hand, is a general-purpose ‘one-size-fits-all’ mobile sensing app designed for digital phenotyping studies. This app takes full advantage of the configurable study protocol and its deployment. Researchers can configure a protocol and deploy it to study participants. The app is pre-loaded with all the available CAMS sampling packages, and a protocol can hence be configured to collect any of the passive and active sensing data, and use any of the connected devices and services available. Part of the app’s user interface is shown in Figure 3, which reflects the domain model of a study protocol.

4 Final Remarks

Recently, mobile sensing as a research methodology has become increasingly challenging, and the use of mobile sensing has been declining [17]. This is mainly rooted in the manufacturers’ aim to reduce background processing (and hence sensing) in order to preserve battery consumption [19] while also becoming increasingly restrictive on what data an app can collect due to privacy concerns. The latter has had the consequence that many mobile sensing apps are not approved and published in the app stores.

CAMS is not immune to these challenges. As a part of the ongoing maintenance and evaluation of CAMS, we run so-called ‘coverage studies’³ which try to gauge the efficiency of passive data sampling (see also [16] for a coverage study of the m-Path Sense app).

However, we would argue that exactly for this reason, it is even more important to embed mobile sensing as part of a domain-specific app and not rely on a generic data sampling app. Moreover, CAMS is designed to accommodate these challenges, especially

³See <https://carp.dk/coverage-analysis-on-android-14-and-ios-18/> for the latest.

through its sampling package architecture, which allows the programmer to only include sensor data permission requests in the Android Manifest and iOS info.plist files, which are actually used in the app. All of this adds to the feasibility of using CAMS in real-world app development.

Acknowledgments

Research on the Copenhagen Research Platform (CARP) is supported by the Copenhagen Center for Health Technology (CACHET), the Innovation Fund, Denmark (IFD), and the Novo Nordisk Foundation (NNF).

References

- [1] Jakob E Bardram. 2019. The cams esense framework: Enabling earable computing for mhealth apps and digital phenotyping. In *Proceedings of the 1st International Workshop on Earable Computing*. 3–7.
- [2] Jakob E Bardram. 2020. The CARP Mobile Sensing Framework—A Cross-platform, Reactive, Programming Framework and Runtime Environment for Digital Phenotyping. *arXiv preprint arXiv:2006.11904* (2020).
- [3] Jakob E Bardram. 2022. Software architecture patterns for extending sensing capabilities and data formatting in mobile sensing. *Sensors* 22, 7 (2022), 2813.
- [4] Jakob E Bardram, Claus Cramer-Petersen, Alban Maxhuni, Mads VS Christensen, Per Bækgaard, Dan R Persson, Nanna Lind, Merete B Christensen, Kirsten Nørgaard, Jayden Khakurel, et al. 2023. DiaFocus: a personal health technology for adaptive assessment in long-term management of Type 2 diabetes. *ACM Transactions on Computing for Healthcare* 4, 2 (2023), 1–43.
- [5] Jakob E Bardram and Mads Frost. 2016. The Personal Health Technology Design Space. *IEEE Pervasive Computing* 15, 2 (2016), 70–78.
- [6] Jakob E. Bardram and Mads Frost. 2017. Double-Loop health technology: Enabling socio-technical design of personal health technology in clinical practice. In *Designing Healthcare That Works: A Sociotechnical Approach*. doi:10.1016/B978-0-12-812583-0.00010-9
- [7] Allan Berrocal, Vlad Manea, Alexandre De Masi, and Katarzyna Wac. 2020. mQoL lab: step-by-step creation of a flexible platform to conduct studies using interactive, mobile, wearable and ubiquitous devices. *Procedia Computer Science* 175 (2020), 221–229.
- [8] Denzil Ferreira, Vassilis Kostakos, and Anind K Dey. 2015. AWARE: mobile context instrumentation framework. *Frontiers in ICT* 2 (2015), 6.
- [9] Syed Monowar Hossain, Timothy Hnat, Nazir Saleheen, Nusrat Jahan Nasrin, Joseph Noor, Bo-Jhang Ho, Tyson Condie, Mani Srivastava, and Santosh Kumar. 2017. mCerebrum: A Mobile Sensing Software Platform for Development and Validation of Digital Biomarkers and Interventions. In *Proceedings of the 15th ACM Conference on Embedded Network Sensor Systems (SenSys '17)*. ACM, New York, NY, USA, 7:1–7:14. doi:10.1145/3131672.3131694
- [10] Devender Kumar, Steven Jeuris, Jakob E Bardram, and Nicola Dragoni. 2020. Mobile and wearable sensing frameworks for mHealth studies and applications: A systematic review. *ACM Transactions on Computing for Healthcare* 2, 1 (2020), 1–28.
- [11] Devender Kumar, Raju Maharjan, Alban Maxhuni, Helena Dominguez, Anne Frølich, and Jakob E Bardram. 2022. mCardia: a context-aware ECG collection system for ambulatory arrhythmia screening. *ACM Transactions on Computing for Healthcare (HEALTH)* 3, 2 (2022), 1–28.
- [12] Nicholas D. Lane et al. 2010. A survey of mobile phone sensing. *IEEE Communications Magazine* 48, 9 (2010). doi:10.1109/MCOM.2010.5560598 arXiv:10 [0163-6804]
- [13] Nicholas D Lane, Mashfiqui Mohammad, Mu Lin, Xiaochao Yang, Hong Lu, Shahid Ali, Afsaneh Doryab, Ethan Berke, Tanzeem Choudhury, and Andrew Campbell. 2011. Bewell: A smartphone application to monitor, model and promote wellbeing. In *5th international ICST conference on pervasive computing technologies for healthcare*. 23–26.
- [14] Neal Lathia, Veljko Pejovic, Kiran K Rachuri, Cecilia Mascolo, Mirco Musolesi, and Peter J Rentfrow. 2013. Smartphones for Large-Scale Behavior Change Interventions. *IEEE Pervasive Computing* 12, 3 (2013), 66–73.
- [15] Merijn Mestdag, Stijn Verdonck, Maarten Piot, Koen Niemeijer, Ghijs Kilani, Francis Tuerlinckx, Peter Kuppens, and Egon Dejonckheere. 2023. m-Path: an easy-to-use and highly tailorable platform for ecological momentary assessment and intervention in behavioral research and clinical practice. *Frontiers in digital health* 5 (2023), 1182175.
- [16] Koen Niemeijer, Merijn Mestdag, Stijn Verdonck, Kristof Meers, Peter Kuppens, et al. 2023. Combining experience sampling and mobile sensing for digital phenotyping with m-path sense: performance study. *JMIR Formative Research* 7, 1 (2023), e43296.
- [17] Veljko Pejović. 2025. Embracing Shifting Trends and Reviving Smartphone Sensing. *IEEE Pervasive Computing* (2025).
- [18] Dan Roland Persson, Jakob E Bardram, and Per Bækgaard. 2024. Perceptions and effectiveness of episodic future thinking as digital micro-interventions based on mobile health technology. *Digital Health* 10 (2024), 20552076241245583.
- [19] Oliver Petter, Marco Hirsch, Eshan Mushtaq, Péter Hevesi, and Paul Lukowicz. 2019. Crowdsensing under recent mobile platform background service restrictions: a practical approach. In *Adjunct Proceedings of the 2019 ACM International Joint Conference on Pervasive and Ubiquitous Computing and Proceedings of the 2019 ACM International Symposium on Wearable Computers*. 793–797.
- [20] Yatharth Ranjan, Zulqarnain Rashid, Callum Stewart, Pauline Conde, Mark Begale, Denny Verbeeck, Sebastian Boettcher, Richard Dobson, and Amos Folarin. 2019. RADAR-Base: Open Source Mobile Health Platform for Collecting, Monitoring, and Analyzing Data Using Sensors, Wearables, and Mobile Devices. *JMIR Uhealth* 7, 8 (01 Aug 2019), e11734. doi:10.2196/11734
- [21] Darius A Rohani, Andrea Quemada Lopategui, Nanna Tuxen, Maria Faurholt-Jepsen, Lars V Kessing, and Jakob E Bardram. 2020. MUBS: A Personalized Recommender System for Behavioral Activation in Mental Health. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. 1–13.
- [22] Stephen M Schueller, Mark Begale, Frank J Penedo, and David C Mohr. 2014. Purple: A Modular System for Developing and Deploying Behavioral Intervention Technologies. *J Med Internet Res* 16, 7 (30 Jul 2014), e181. doi:10.2196/jmir.3376
- [23] John Torous, Mathew V Kiang, Jeanette Lorme, and Jukka-Pekka Onnela. 2016. New tools for new research in psychiatry: a scalable and customizable platform to empower data driven smartphone research. *JMIR mental health* 3, 2 (2016), e16.
- [24] John Torous, Hannah Wisniewski, Bruce Bird, Elizabeth Carpenter, Gary David, Eduardo Elejalde, Dan Fulford, Synthia Guimond, Ryan Hays, Philip Henson, Liza Hoffman, Chun Lim, Michael Menon, Valerie Noel, John Pearson, Randy Peterson, Ammu Susheela, Haley Troy, Aditya Vaidyam, Emma Weizenbaum, John A Naslund, and Matcheri Keshavan. 2019. Creating a Digital Health Smartphone App and Digital Phenotyping Platform for Mental Health and Diverse Healthcare Needs: an Interdisciplinary and Collaborative Approach. *Journal of Technology in Behavioral Science* 4, 2 (2019), 73–85. doi:10.1007/s41347-019-00095-w
- [25] Rui Wang, Fanglin Chen, Zhenyu Chen, Tianxing Li, Gabriella Harari, Stefanie Tignor, Xia Zhou, Dror Ben-Zeev, and Andrew T Campbell. 2014. StudentLife: assessing mental health, academic performance and behavioral trends of college students using smartphones. In *Proceedings of the 2014 ACM International Joint Conference on Pervasive and Ubiquitous Computing*. ACM, 3–14. doi:10.1145/2632048.2632054
- [26] Haoyi Xiong, Yu Huang, Laura E Barnes, and Matthew S Gerber. 2016. Sensus: a cross-platform, general-purpose system for mobile crowdsensing in human-subject studies. In *Proceedings of the 2016 ACM International Joint Conference on Pervasive and Ubiquitous Computing*. ACM, 415–426.